回声档案

校园线索推理与多结局探索应用

课程名称	鸿蒙高级应用开发工程师认证
学　　院	信息工程学院
专　　业	物联网工程（华为班）
班　　级	23 物联 1 班
学生姓名	傅哲
学　　号	20230313204
指导教师	王　旗
成　　绩	
开课时间	2026—2027 学年　一　学期

期 末 大 作 业 报 告

GitHub：Nekozhe　|　https://github.com/Nekozhe/EchoArchive

评 分 标 准

指标	评价内容	评价标准	分值
目标一 （16 分）	需求与问题分析	说明用户需求、主要功能及范围	5
	数据与状态分析	说明关键数据、状态变化及模块间数据流	6
	边界与异常分析	分析至少两种边界或异常及处理方式	5
目标二 （28 分）	架构与模块设计	说明整体架构、关键模块接口以及本人负责内容	8
	核心功能实现	完成选题要求的可运行功能和完整业务流程	12
	页面与交互体验	页面布局合理、交互过程顺畅，操作结果反馈清晰	8
目标三 （36 分）	ArkTS 与 ArkUI 开发	正确使用组件、状态管理和开发工具，工程可运行	9
	数据持久化与异常处理	完成数据保存、进程重启恢复与异常处理	9
	单元测试与 Mock	真实运行测试；每人至少 3 条，覆盖正常、边界、异常	12
	GitHub 贡献与项目交付	个人 Commit 可核验，仓库有源码、说明、截图和演示	6
目标四 （20 分）	新技术认识与适用局限	结合可追溯资料说明其用途、适用版本及局限	8
	自主学习与资料分析	围绕一个技术疑问，列出至少两份参考资料与结论	8
	持续学习与更新计划	提出具体学习任务与资料核查、成效验证办法	4
独立完成	独立完成奖励	仅独立完成者可申请，按综合能力核定	0—10

表 0.1　期末大作业评分标准（成绩由评阅教师填写）

一、项目概述

1.1 基本信息

项目要素	填写内容
作品名称	回声档案

项目要素	填写内容
指定主题	主题一：迷雾档案 —— 校园线索推理与多结局探索
姓名（学号）	傅哲（20230313204）
GitHub 账号	Nekozhe
GitHub 仓库链接	https://github.com/Nekozhe/EchoArchive
小组成员	无（独立完成）
开发环境	DevEco Studio 26.0.0.821；HarmonyOS SDK 6.1.1(24)；targetSdkVersion 26.0.0；API 26 模拟器（Mate 90 Pro）

表 1.1　项目基本信息

1.2 项目简介

《回声档案》是一款运行在 HarmonyOS 上的单机校园推理应用。答辩前夜，实验室的竞赛原型机被搬走，机箱第二天出现在天台，主板和硬盘不见了。玩家以「受辅导员委托的临时调查员」身份介入，在三个场景中探索取证、盘问三名关系人，把线索排成一条能站住的证据链，最终作出指认并进入对应结局。

选择这个题材，是因为它给「状态」提供了一个自然的载体：玩家不是在填表，而是在不断缩小嫌疑范围。每拿到一条线索，可触发的节点、可追问的对话、可选择的物证都会随之变化，而这些变化必须被正确保存——否则玩家中途退出就得从头再来。这正好把课程中的状态管理、数据持久化与单元测试三块内容串成一条线。

与常见做法不同，本作品没有把推理做成「三选一按钮」：玩家要同时给出人物、时间段、关键物证三项，由判定函数检查证据链是否自洽。指对人但证据不成立，会进入「断链」结局；指错人则进入「替罪羊」结局。这样推理才有反馈，而不是猜。

1.3 需求完成情况

功能	实现情况	状态
委托与开场	首页展示案件简报与进度卡，可开始新档或继续上次调查	已完成
场景探索	3 个场景，14 个事件节点，节点带前置条件与状态标记	已完成
线索收集	8 条线索（5 条关键），线索板支持未解锁态与详情浮层	已完成
人物对话	5 段对话，逐句播放并配人物立绘，出示线索可解锁新节点	已完成

功能	实现情况	状态
推理判定	人物 / 时段 / 物证三槽位校验，三种结局分支	已完成
数据持久化	关键状态自动写入 Preferences，杀进程后完整恢复	已完成
单元测试	6 条用例覆盖正常 / 边界 / 异常，模拟器实测通过	已完成
运行录屏	1 分 35 秒，含探索、推理、结局与重启恢复	已完成

表 1.2　需求完成情况

尚未完成的部分：原计划为场景添加可点击的热区（点击画面中的具体位置取证），当前以节点卡片列表代替；此外路由仍使用已标记废弃的 router 接口，计划后续迁移至 Navigation 组件（详见 6.3 节）。

二、系统架构

2.1 整体架构



图 2.1　系统架构图（全部模块由本人设计与实现）

应用采用三层结构：页面层只负责渲染与收集用户操作；业务层是全部规则判断，由纯函数组成，不依赖 UI 与存储；数据层分为只读的案情常量表与可写的玩家进度存档。

以「调看监控回放」这一操作为例：场景页把当前 GameState 与节点定义传给 canTrigger()，函数依次检查该节点是否已执行、门禁记录 C2 是否已获得、管理员信任度是否不低于 1；通过后 applyEvent() 返回一份新的 GameState（追加线索 C4、记录节点编号），存档服务把它序列化写入 Preferences，页面再用新状态刷新线索计数与节点标记。整个过程中页面不直接修改数据，所有变化都经由业务层返回新对象。

2.2 模块划分

模块	主要职责	输入 / 输出	负责人
pages/（5 个页面）	委托页、场景探索、线索板、推理台、结局页的渲染与交互	用户操作 → 界面刷新	本人
logic/GameLogic	门控判定、事件生效、推理判定，全部为纯函数	GameState + 节点 → 新 GameState / 判定结果	本人
data/CaseData	线索、事件节点、对话、结局、推理选项的静态数据	只读常量	本人
model/GameTypes	数据类型与存档结构定义	被各层引用	本人
service/SaveService	Preferences 读写、版本校验、异常降级	GameState ↔ JSON 字符串	本人
ohosTest/GameLogic.test	6 条业务规则单元测试	构造状态 → 断言结果	本人

表 2.1　系统模块及接口关系

2.3 本人职责

本作品为独立完成，页面层、业务层、数据层的全部模块均由本人设计与实现，包括案情数据编写、五个页面的布局与交互、存档服务、单元测试与文档。

过程中有三个关键设计决定：一是把推理规则从页面里抽出来做成纯函数，使同一份规则既能被界面调用也能被测试直接断言；二是把剧情推进抽象为「前置条件 + 产出物」的数据表，新增剧情只需改数据、不改页面代码；三是给存档加 schemaVersion，读取时结构不符就降级为新档，避免旧档导致启动崩溃。

三、模块设计

3.1 功能设计



图 3.1　核心业务流程图

下面以「一次完整的调查与指认」为主线说明业务流程。玩家接受委托后选择场景，点击事件节点时先经过门控：不满足前置条件就提示具体原因（缺哪条线索、信任度不足），节点保持受限；满足则执行并发放线索，同时自动存档。线索累计到 4 条时推理台解锁，玩家在三个槽位作出选择，提交后由判定函数给出结局。

流程中三个判定点对应三条真实规则：门控判定决定剧情能否推进，解锁判定决定推理台是否可用，提交校验决定这次指认是否有效。三者都返回具体原因而不是简单地禁用按钮，这是让玩家「知道自己该去做什么」的关键。

3.2 数据设计

类 / 数据对象	关键字段与类型	业务含义	存储与读取方式
Clue	id: string, name, desc, keyClue: boolean, sceneId	线索卡内容与归属场景	源码常量表，只读
EventNode	requires: string[], requiresAny: string[], requireTrust: number, grantClues: string[]	事件节点的门控条件与产出	源码常量表，只读
GameState	ownedClues: string[], doneEvents: string[], trust: number, attemptCount: number, endingId: string, schemaVersion: number	玩家进度，存档主体	Preferences，JSON 序列化
DeductionSlots	person: string, period: string, evidence: string	一次推理的三个槽位	运行时内存
JudgeResult	ok: boolean, endingId: string, reason: string	判定结果与失败原因	运行时内存

表 3.1　主要数据模型

3.3 接口设计

接口 / 方法	输入参数	返回值	异常 / 边界处理
canTrigger(state, node)	GameState, EventNode	boolean	节点已执行、线索不足、信任度不足均返回 false
applyEvent(state, node)	GameState, EventNode	新的 GameState	入参只读，返回副本，避免副作用
judgeDeduction(state, slots)	GameState, DeductionSlots	JudgeResult	槽位为空 / 物证未获得 / 物证不存在分别给出原因
SaveService.load()	无	Promise<GameState>	无存档、JSON 非法、版本不符均返回新档
SaveService.save(state)	GameState	Promise<void>	写入失败只记录日志，不阻断界面
SaveService.parse(raw)	string	GameState	纯函数，便于脱离设备做单元测试

表 3.2　模块接口

四、代码实现与贡献

4.1 核心代码

推理判定是整个作品的核心：它把「玩家认为是谁干的」翻译成一次可以验证的计算。代码位于 entry/src/main/ets/logic/GameLogic.ets，方法名 judgeDeduction()。

```
export function judgeDeduction(state: GameState, slots: DeductionSlots): JudgeResult {
  if (slots.person === '' || slots.period === '' || slots.evidence === '') {
    return { ok: false, endingId: '', reason: '假设不完整：人物、时段、物证都要选' };
  }
  if (findClue(slots.evidence) === undefined) {
    return { ok: false, endingId: '', reason: '这条物证不存在' };
  }
  if (!hasClue(state, slots.evidence)) {
    return { ok: false, endingId: '', reason: '这条物证你还没有拿到' };
  }
  if (slots.person !== 'A') {
    return { ok: true, endingId: 'END3', reason: '你作出了指认' };
  }
  if (slots.period === 'P2' && slots.evidence === 'C7') {
    return { ok: true, endingId: 'END1', reason: '证据链完整' };
  }
  return { ok: true, endingId: 'END2', reason: '指认成立，但证据链不完整' };
}
```

代码 4.1　推理判定函数（logic/GameLogic.ets）

这段代码的价值在于顺序：先查完整性，再查物证是否属于本人，最后才判断人物与证据链。如果调换顺序，玩家即使一条线索都没有，也能通过不断试错把三个结局刷出来。函数只读入参、不修改状态，返回新的结果对象，因此可以被单元测试反复调用而互不干扰。

4.2 技术应用

状态管理使用 @State 与 @Builder 组合：页面持有 GameState 的副本，每次业务函数返回新对象后整体替换，触发界面刷新；列表与选项用 ForEach 渲染，靠 id 做键避免复用错位。这里踩过一个坑：@Builder 的传值参数不会随状态变化刷新，导致选项点击后不高亮，最终改为在 @Builder 内直接读取组件状态（见 5.3 节）。

页面跳转使用 router 的 pushUrl / replaceUrl：结局页用 replaceUrl 而非 pushUrl，避免用户返回时又回到已提交的推理台。数据持久化使用 Preferences 以 JSON 形式存整份进度，原因是本作的数据结构是一棵整体读写的树，没有按字段查询的需求，键值存储比关系型数据库更轻。界面样式统一由 common/Theme.ets 提供色值，配合 linearGradient 做图片到背景的过渡，保证深色主题下文字始终可读。

4.3 Git 提交记录

Commit 短哈希	提交说明	涉及文件 / 功能
01e1fc1	chore: 初始化仓库与项目骨架	.gitignore / README / 方案提案
3c36c10	chore: 接入 DevEco Stage 工程骨架	工程配置、EntryAbility、首页
a5eefdd	docs: 修正 README 中的 SDK 版本	README.md
9f994b1	feat: 实现完整调查流程与本地存档	model / data / logic / service / 5 个页面
6d41f0c	test: 补充 6 条核心业务规则单元测试	ohosTest/GameLogic.test.ets
ca7d841	docs: 添加软件需求说明书	docs/02-需求说明书.docx
fb7b96b	feat: 接入美术素材并统一深色系统栏	media 资源、场景页、对话、系统栏
8af249f	fix: 修复推理台选中态不刷新与假设摘要显示内部 id	pages/DeductionPage.ets
0f78884	docs: README 补充运行截图、录屏与测试说明	README、docs/images、docs/video
3bfa4a0	docs: 更新运行录屏	docs/video/demo.mp4

表 4.1　本人 Git 提交记录（共 10 次，均为本人账号 Nekozhe 提交）

五、模块测试

5.1 Mock 测试设计

被测对象是业务层的纯函数，它们只依赖 GameState 与静态数据，不依赖界面与设备，因此不需要框架级 Mock：直接构造不同形态的 GameState 作为输入即可。测试放在 entry/src/ohosTest/ets/test/GameLogic.test.ets，使用 Hypium 的 describe / it / expect。

六条用例按「正常—边界—异常」分布：正常两条验证主流程与三种结局分支；边界两条分别验证线索前置与信任度门槛；异常两条验证推理槽不完整与存档损坏。断言尽量落在返回值与状态字段上，而不是界面文案，这样界面改版不会导致测试失效。

5.2 测试结果

类别	Mock 输入 / 前置条件	预期结果	实际
正常	新档 + 执行节点 E01（检查工作台）	节点可触发，获得 C1，doneEvents 含 E01	通过
边界	无 C6 时请求 E08；先执行 E07 再请求	前者不可触发，后者可触发，执行后 once 生效	通过

类别	Mock 输入 / 前置条件	预期结果	实际
边界	仅有 C2（信任度 0）时请求 E05	不可触发；执行 E04 后信任度为 1，转为可触发	通过
异常	空槽位 / 未获得的 C7 / 不存在的 C99	三种输入均返回 ok=false 并给出原因	通过
正常	A+P2+C7 / A+P1+C7 / B+P2+C7	分别判定为 END1 / END2 / END3	通过
异常	空串、非法 JSON、缺字段、版本号 99、合法存档	前四种降级为新档，合法存档完整往返	通过

表 5.1　模块测试记录

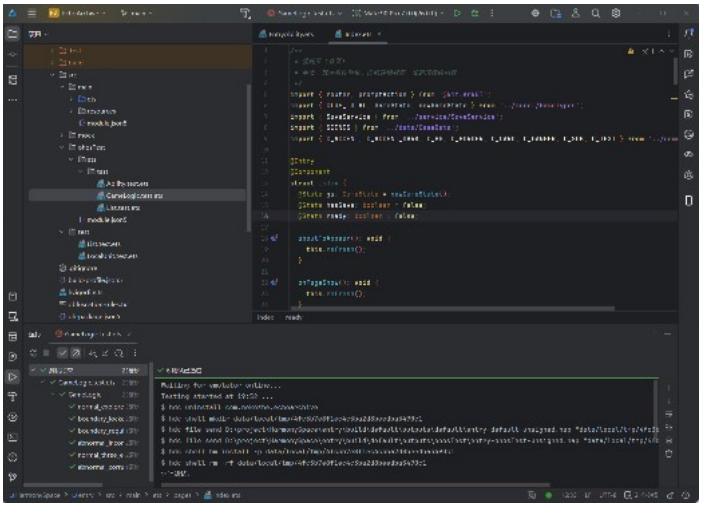


图 5.1　单元测试在 API 26 模拟器上的实际运行结果（6 条全部通过）

测试包与主包分开安装（entry-ohosTest-unsigned.hap），运行日志可见 hdc shell bm install 的完整安装过程，说明测试是在真实设备环境中执行的，而非本地静态检查。

5.3 问题与修复

开发中最典型的一个问题出现在推理台：选项点击后，假设摘要能正确更新，但被选中的选项本身始终没有高亮，看起来像没点上。

排查过程：先确认点击事件确实触发了（在 onClick 里打印日志，状态值每次都在变），说明问题不在事件绑定；再把选项的 color 直接写死成固定值，界面能正常变色，说明渲染链路没问题；最后定位到选项是用 @Builder 封装的，当前选中项通过参数传入，而 @Builder 的值参数在状态变化时不会重新求值，界面拿到的始终是首次渲染时的空值。

修复方式：不再把选中项作为参数传入，改为在 @Builder 内部调用组件方法 selectedOf(kind) 直接读取 @State，同时把假设摘要从显示内部 id 改为显示选项标签。修改后重新构建、安装到模拟器验证：三个槽位点击后均正确高亮，提交后按预期进入结局一，回归测试六条用例仍全部通过。

六、运行与总结

6.1 运行效果

下面按一条真实操作流程展示运行效果：打开应用进入委托页，读案件简报后接受委托；在 302 实验室依次检查工作台、门禁面板与垃圾桶，每次获得线索都会弹出提示；转到值班室与管理员对话，取得信任后才能调看监控；线索累计 8 条后进入推理台，选定人物、时段与物证并提交，进入结局一。



（a）委托页



（b）场景探索



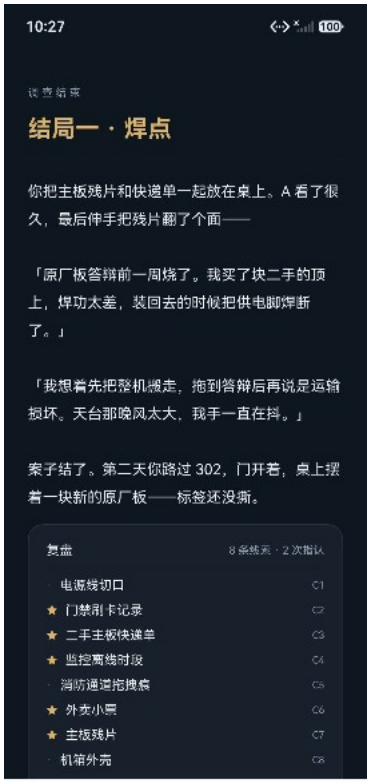
（c）人物对话



（d）线索板



（e）推理台



（f）结局一

图 6.1　应用运行效果（API 26 模拟器实测截图）

6.2 数据恢复与集成

持久化的验证方式是「杀进程 + 重开」：在完成一次指认后，从后台划掉应用，再从桌面图标重新启动，首页的进度卡显示 8/8 条线索、10 个节点已调查、已完成一次指认，进入场景后各节点的已完成状态与线索板内容完全一致，无需重新调查。



图 6.2　杀进程重启后的状态恢复结果

实现上，存档在三个时机写入：执行事件节点后、获得线索后、提交指认后，不依赖页面销毁时机，因此异常退出也不会丢失已完成的调查。存档带 schemaVersion 字段，读取时若 JSON 非法、字段缺失或版本不一致，一律降级为新档并正常启动，这条策略由 6 号单元测试用四种输入覆盖。

6.3 个人总结

这个项目最初被我当成「写几个页面 + 存个档」，真正做下来才发现难点在别处：怎么让剧情推进由规则驱动而不是由 if 堆出来。最后我把每个事件节点都定义成「前置条件 + 产出物」的数据，页面只负责渲染，规则全部收进纯函数，这才让「线索真实改变后续」这件事变得可验证——同一份规则，界面能调，测试也能调。

开发中遇到的第一个技术疑问是：单机应用保存进度，该用 Preferences 还是关系型数据库 RDB？我查了两份资料：华为开发者联盟的《应用数据持久化》概述（developer.huawei.com，对应 HarmonyOS 6.1.1 文档）说明 Preferences 适合存放少量键值型配置，RDB 适合需要按条件查询的结构化数据；另一份是 Preferences 开发指南，其中明确其容量与并发能力的适用边界。结合本作的实际需求——进度是一棵整体读写的树、没有按字段检索的场景、单用户单进程——我选择 Preferences 以 JSON 存整份进度，并在外层加 schemaVersion 做版本管理。它的局限也很清楚：不能做部分字段更新，数据量大时整体序列化的开销会上升，多进程并发写也没有事务保证。如果后续要给线索加上标签检索或统计，就应该迁移到 RDB。

第二个疑问来自界面层：为什么 @Builder 里传入的选中项不刷新？我对照官方《ArkTS 状态管理》与 @Builder 装饰器两处文档，确认值参数传递不会建立观察关系，只有 @State / @Prop / @Link 这类装饰的变量才会驱动重建。这也让我理解了 ArkUI 声明式更新的前提：界面要「看得见」状态，状态必须被框架观察到。修复后我把这条经验写进了 5.3 节，因为它比任何一次功能实现都更能说明「框架约定」和「直觉写法」之间的差距。

下一步我给自己定了两个具体任务。第一，把 router 迁移到 Navigation 组件：当前 router 在 API 26 已标记废弃，构建时会产生二十多条告警。验证办法是先用 DevEco 的 API 差异报告确认 Navigation 的推荐用法，再逐个页面替换，最后跑一遍六条单元测试与完整流程回归，确认页面栈行为一致。第二，给场景页加上可点击热区，让取证从「点卡片」变成「点画面」，这需要学习 Canvas 绘制与手势命中测试。资料核查上，我会优先看官方 API 文档与版本变更说明，遇到与博客冲突时以官方为准，并在提交信息里记录依据。